

Operating Systems Study Guide

MEMORY MANAGEMENT & DISK SCHEDULING Q&A

1. Explain First-Fit, Best-Fit, and Worst-Fit Memory Allocation with Example.

Memory allocation is the process of assigning free memory blocks to processes. First-Fit, Best-Fit, and Worst-Fit are common strategies used for contiguous memory allocation.

First-Fit: In First-Fit, the operating system allocates the first free memory block that is large enough to satisfy the process request.

Example: Free blocks: 100 KB, 500 KB, 200 KB, 300 KB. Process size = 180 KB. The first block large enough is 500 KB, so the process is allocated there.

Best-Fit: In Best-Fit, the smallest free block that can accommodate the process is selected.

Example: Free blocks: 100 KB, 500 KB, 200 KB, 300 KB. Process size = 180 KB. The smallest suitable block is 200 KB, so the process is allocated there.

Worst-Fit: In Worst-Fit, the largest available free block is selected.

Example: Free blocks: 100 KB, 500 KB, 200 KB, 300 KB. Process size = 180 KB. The largest block is 500 KB, so the process is allocated there.

Comparison: First-Fit is fast and simple. Best-Fit minimizes wasted space initially. Worst-Fit leaves large free blocks available for future allocations.

2. With Suitable Diagram Explain Swapping.

Swapping is a memory-management technique in which a process is temporarily moved from main memory to secondary storage (backing store) and later brought back into memory for execution.

Working of Swapping: When memory becomes full, the operating system transfers some processes from main memory to backing store. When the process is needed again, it is swapped back into memory and execution continues.

[Diagram: Swapping Processes between Main Memory (RAM) and Backing Store (Disk)]

Advantages: Increases degree of multiprogramming. Improves memory utilization. Allows more processes to execute than can fit in memory simultaneously.

Disadvantages: Swapping causes overhead. Excessive swapping reduces system performance.

3. Explain Contiguous Memory Allocation.

In Contiguous Memory Allocation, each process is allocated a single continuous block of memory. The operating system divides memory into partitions and assigns one partition to each process.

Types of Contiguous Allocation:

- **Fixed Partition Allocation:** Memory is divided into fixed-size partitions. Each partition can hold only one process.
- **Variable Partition Allocation:** Partitions are created dynamically according to process requirements.

[Diagram: Contiguous Memory Layout (OS, Process P1, Process P2, Free Space, Process P3)]

Advantages: Simple implementation. Easy memory management.

Disadvantages: External fragmentation may occur. Difficult to utilize memory efficiently.

4. Explain Paging Hardware.

Paging Hardware supports paging by translating logical addresses into physical addresses using a page table.

Logical Address: Generated by the CPU and divided into Page Number (p) and Offset (d).

Page Table: The page table stores the mapping between page numbers and frame numbers.

Address Translation: The CPU generates a logical address. The page number is used as an index into the page table to obtain the frame number. The frame number is combined with the offset to generate the physical address.

[Diagram: CPU Logical Address Page Table Indexing Frame Translation Main Memory]

Advantage: Paging hardware eliminates external fragmentation and enables efficient memory utilization.

5. Explain Paging Memory Management with an Example.

Paging is a memory management technique that divides logical memory into fixed-size pages and physical memory into fixed-size frames.

Working: When a process is loaded, its pages are placed into available frames in physical memory. A page table keeps track of the mapping between pages and frames.

Example: Page Size = 1 KB.

Page Table: Page 0 → Frame 5, Page 1 → Frame 9, Page 2 → Frame 3.

Suppose the logical address is (Page = 2, Offset = 100). From the page table, Page 2 maps to Frame 3. Therefore, Physical Address = Frame 3 + Offset 100.

Advantages: No external fragmentation. Efficient memory allocation. Supports virtual memory.

6. Explain Segmentation with a Neat Diagram.

Segmentation is a memory-management technique that divides a program into logical units called segments. Each segment represents a logical entity such as code, data, stack, or procedures.

Segment Table: A segment table stores the Base Address and Limit for each segment.

[Diagram: Logical Address (Segment, Offset) Segment Table Physical Memory Translation]

Advantages: Supports logical program structure. Easy sharing and protection of memory. Reduces program complexity.

Disadvantages: Causes external fragmentation. Segment table management is required.

7. Explain Segmentation Hardware.

Segmentation hardware translates logical addresses into physical addresses using a segment table.

Logical Address: A logical address consists of a Segment Number and an Offset.

Segment Table: Each segment table entry contains a Base Address and a Limit.

Address Translation: The segment number selects an entry from the segment table. The offset is checked against the limit value. If $\text{offset} < \text{limit}$, the physical address is generated as: $\text{Physical Address} = \text{Base} + \text{Offset}$.

[Diagram: Segmentation Address Hardware Protection Check ($\text{Offset} < \text{Limit}$)]

Benefit: Segmentation hardware provides protection and sharing at the segment level.

8. Write a Short Note on Fragmentation.

Fragmentation is the wastage of memory caused by inefficient memory allocation. It reduces memory utilization and affects system performance.

Internal Fragmentation: Occurs when allocated memory is larger than the requested memory. *Example:* A process requires 18 KB but receives a 20 KB partition. The unused 2 KB causes internal fragmentation.

External Fragmentation: Occurs when free memory is available but scattered into small non-contiguous blocks. Although total free memory is sufficient, it cannot satisfy large requests.

Solutions: Compaction, Paging, Segmentation with Paging.

9. Explain Segmentation with Paging.

Segmentation with Paging combines the advantages of both segmentation and paging. The program is first divided into segments, and each segment is further divided into pages.

Working: Each segment has its own page table. The logical address contains a Segment Number, Page Number, and Offset.

Address Translation Steps: 1. Segment number identifies the segment. 2. Page number identifies the page within that segment. 3. Offset identifies the location within the page.

[Diagram: Combined Architecture (Logical Address Segment Table Page Table Main Memory)]

Advantages: Eliminates external fragmentation. Supports logical program organization. Provides protection and sharing.

10. Write a Short Note on Demand Paging.

Demand Paging is a virtual memory technique in which pages are loaded into memory only when they are required. Instead of loading the entire process, only the needed pages are brought into memory.

Working: When a process accesses a page that is not in memory, a page fault occurs. The operating system loads the required page from secondary storage into a free frame and resumes execution.

Steps in Demand Paging: 1. CPU references a page. 2. Page fault occurs. 3. Operating system locates the page on disk. 4. Page is loaded into memory. 5. Page table is updated. 6. Execution continues.

Advantages: Reduces memory usage. Faster program startup. Supports large virtual memory.

Disadvantages: Page faults increase overhead. Excessive page faults may reduce performance.

11. Write Short Notes on Disk Management.

Disk management is the process of organizing, controlling, and maintaining disk storage efficiently. The operating system performs disk management to ensure efficient storage and retrieval of data.

- **Disk Formatting:** Before a disk can be used, it must be formatted. Formatting divides the disk into tracks and sectors and prepares it for storing data.
- **Partitioning:** A disk can be divided into multiple logical sections called partitions. Each partition can contain a separate file system.
- **Boot Block:** The boot block contains the bootstrap program used to start the operating system when the computer is powered on.
- **Bad Block Recovery:** Some disk sectors may become defective. The operating system identifies and replaces bad blocks with spare sectors.

- **Free Space Management:** The operating system keeps track of free disk space and allocates it to files when needed.

Advantages: Efficient storage utilization. Faster file access. Better reliability and performance.

12. With a Neat Diagram Explain Dynamic Relocation Using Relocation Register.

Dynamic relocation allows a program to be loaded into any location in memory during execution. It is achieved using a relocation register.

Relocation Register: The relocation register contains the starting physical address of the process in memory.

Working: The CPU generates a logical address. The value in the relocation register is added to the logical address to obtain the physical address.

Formula: $\text{Physical Address} = \text{Logical Address} + \text{Relocation Register}$

Example: Relocation Register = 14000, Logical Address = 346. $\text{Physical Address} = 14000 + 346 = 14346$.

[Diagram: $\text{Logical Address} + \text{Relocation Register} = \text{Physical Address}$]

Advantages: Programs can be loaded anywhere in memory. Supports multiprogramming. Improves memory utilization.

13. Explain FIFO Page Replacement Policy.

FIFO (First-In First-Out) is the simplest page replacement algorithm. The page that entered memory first is removed first when a page fault occurs.

Working: Pages are maintained in a queue. When memory becomes full and a new page is required, the oldest page is replaced.

Example: Reference String: 1, 2, 3, 4. Frames = 3. 1 [1], 2 [1 2], 3 [1 2 3], 4 Replace 1. After page 4 arrives, page 1 is removed because it entered first.

Advantages: Easy to implement. Low overhead.

Disadvantages: May replace frequently used pages. Suffers from Belady's anomaly.

14. Explain LRU Page Replacement Algorithm with an Example.

LRU (Least Recently Used) replaces the page that has not been used for the longest period of time.

Working: The operating system keeps track of page usage. When a replacement is required, the least recently used page is removed.

Example: Reference String: 1, 2, 3, 1, 4. Frames = 3. 1 → [1], 2 → [1 2], 3 → [1 2 3], 1 → [1 2 3], 4 → Replace 2. Page 2 is replaced because it was used least recently.

Advantages: Better performance than FIFO. Utilizes recent page usage information.

Disadvantages: More complex implementation.

15. Page Fault Calculation Summary.

Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1. Frames = 3.

Algorithm	Page Faults
FIFO	15
LRU	12
Optimal	9

16. Explain Optimal Page Replacement Algorithm.

Optimal Page Replacement replaces the page that will not be used for the longest period in the future. It produces the minimum possible number of page faults and is used as a benchmark.

Working: Whenever a page fault occurs, the operating system looks ahead in the reference string and replaces the page whose next use is farthest away.

Example: Frames = 3. Reference String: 1, 2, 3, 4. When page 4 arrives, the page whose future use is farthest away is replaced.

Advantages: Minimum page faults. Best possible performance.

Disadvantages: Future references are unknown. Cannot be implemented practically.

17. Write a Note on Allocation of Frames.

Frame allocation is the process of distributing available memory frames among processes.

- **Minimum Number of Frames:** Each process must be allocated a minimum number of frames required for execution.
- **Equal Allocation:** All processes receive an equal number of frames (e.g., 100 frames / 5 processes = 20 frames each).
- **Proportional Allocation:** Frames are allocated according to process size. Larger processes receive more frames.

- **Priority Allocation:** Frames are allocated based on process priority. Higher-priority processes receive more frames.

Advantages: Efficient memory utilization. Reduces page faults. Improves system performance.

18. Write a Note on Thrashing.

Thrashing is a condition in which the operating system spends more time paging than executing processes. It occurs when insufficient frames are allocated to processes, resulting in excessive page faults.

Causes: High degree of multiprogramming, insufficient physical memory, excessive page faults.

Effects: Low CPU utilization, reduced system performance, increased response time.

Prevention: Reduce multiprogramming level, use Working-Set Model, use Page Fault Frequency technique.

More Page Faults → More Paging → Less CPU Usage → Thrashing

19. Write a Note on Working-Set Model.

The Working-Set Model is used to prevent thrashing by keeping track of the pages actively used by a process.

Working Set: The working set of a process is the collection of pages referenced during a recent period of execution.

Working Set Window (Δ): A fixed number of recent memory references is considered. Pages referenced within this window form the working set.

Working: If all pages in the working set are present in memory, the process executes efficiently with fewer page faults.

Advantages: Reduces thrashing. Improves CPU utilization. Helps determine frames required.

20. Detailed FIFO Table Calculation.

Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1. Number of Frames = 3.

Allocations follow standard FIFO substitution. Total Page Faults = 15.

21. Page Fault Calculation String 2.

Reference String: 7, 2, 3, 1, 2, 5, 3, 4, 6, 7, 7, 1, 0, 5, 4, 6, 2, 3, 0, 1. Frames = 3.

Algorithm	Page Faults
FIFO	17
LRU	18
Optimal	12

22. Page Fault Calculation String 3.

Reference String: 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6. Frames = 4.

Algorithm	Page Faults
LRU	10
FIFO	11
Optimal	8

23. Explain First Come First Serve (FCFS) Disk Scheduling with Example.

FCFS is the simplest disk scheduling algorithm. Requests are serviced in the order in which they arrive in the disk queue.

Working: The disk arm moves from one request to the next according to arrival order without considering seek time.

Example: Request Queue: 98, 183, 37, 122. Initial Head Position = 53.

Order: 53 → 98 → 183 → 37 → 122.

Total Head Movement Calculation: $|98-53| + |183-98| + |37-183| + |122-37| = 45 + 85 + 146 + 85 = 361$ tracks.

Advantages: Simple and fair. Easy to implement.

Disadvantages: High seek time.

24. Explain Shortest Seek Time First (SSTF) Disk Scheduling with Example.

SSTF selects the request closest to the current head position to minimize head movement.

Example: Request Queue: 98, 183, 37, 122. Initial Head Position = 53.

Execution Order: 53 → 37 → 98 → 122 → 183.

Total Head Movement: $16 + 61 + 24 + 61 = 162$ tracks.

Advantages: Reduces seek time. Better performance than FCFS.

Disadvantages: Starvation may occur.

25. Explain SCAN Disk Scheduling with Example.

SCAN is also called the Elevator Algorithm. The disk arm moves in one direction servicing requests and then reverses direction.

Example: Request Queue: 98, 183, 37, 122, 14, 124. Initial Head = 53. Direction = Towards 0.

Order: 53 → 37 → 14 → 0 → 98 → 122 → 124 → 183.

Advantages: Provides better response time. Reduces starvation.

Disadvantages: Boundary delays.

26. Explain C-SCAN Disk Scheduling with Example.

C-SCAN (Circular SCAN) services requests in only one direction, returning to the beginning without servicing requests during the return trip.

Example: Request Queue: 98, 183, 37, 122, 14, 124. Initial Head = 53. Direction = Towards Higher Tracks.

Order: 53 → 98 → 122 → 124 → 183 → 199 [Jump to 0] → 14 → 37.

Advantages: Provides uniform waiting time. Fairer than SCAN.

27. Explain LOOK Disk Scheduling with Example.

LOOK is similar to SCAN but does not move to the physical end of the disk unless a request exists there.

Example: Request Queue: 98, 183, 37, 122, 14, 124. Initial Head = 53. Direction = Towards Lower Tracks.

Order: 53 → 37 → 14 → 98 → 122 → 124 → 183.

Advantages: Less head movement than SCAN. Better efficiency.

28. Explain C-LOOK Disk Scheduling with Example.

C-LOOK is the circular version of LOOK. The head services requests in one direction only, jumping back to the lowest request after reaching the highest request.

Example: Request Queue: 98, 183, 37, 122, 14, 124. Initial Head = 53. Direction = Towards Higher Tracks.

Order: 53 → 98 → 122 → 124 → 183 [Jump to 14] → 37.

Advantages: Uniform waiting time. Less head movement than C-SCAN.

29. Disk Scheduling Numerical 1.

Given: Tracks = 200 (0-199). Queue: 55, 58, 39, 18, 90, 160, 150, 38, 184. Initial Head = 100.

Algorithm	Average Seek Length
FIFO	55.33 tracks
SSTF	27.56 tracks
SCAN (Towards 0)	31.56 tracks
C-SCAN (Towards High)	43.11 tracks

30. Disk Scheduling Numerical 2.

Given: Queue: 98, 183, 37, 122, 14, 124, 65, 67. Initial Head = 55.

Algorithm	Average Seek Length
FIFO	79.75 tracks
SSTF	29.25 tracks
LOOK (Towards High)	37.13 tracks
C-LOOK	40.00 tracks